

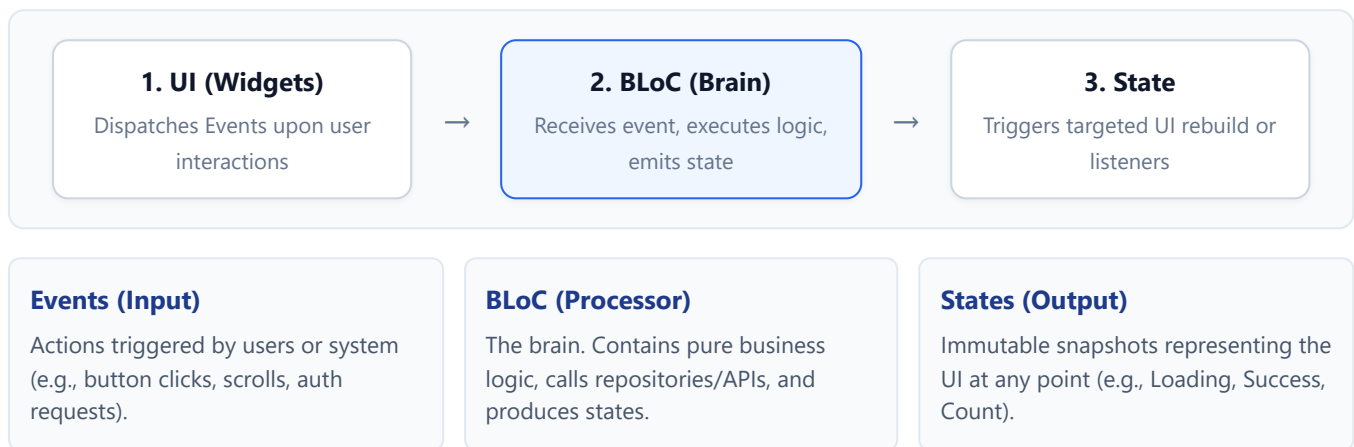
FLUTTER ARCHITECTURAL GUIDE

State Management Masterclass: BLoC Pattern

Complete guide covering theoretical foundation, Equatable value equality, Flutter widgets, and implementation.

1. The Core Mental Model

BLoC stands for **Business Logic Component**. It separates presentation from business logic using reactive **Dart Streams** with a strict unidirectional data flow.



2. Why Equatable is Essential

By default, Dart checks **reference equality** (memory addresses). Two different class instances with identical properties evaluate as unequal:

```
CounterState(1) == CounterState(1); // false without Equatable!
```

With `Equatable`, Dart performs **value equality** based on the fields listed in `props`:

```
CounterState(1) == CounterState(1); // true with Equatable!
```

Why BLoC Requires This

BLoC skips UI rebuilds if a newly emitted state is identical to the current state. Without `Equatable`, BLoC would needlessly rebuild widgets even when the underlying data has not changed.

3. Step-by-Step Implementation

Step 1: Define Events

lib/bloc/counter_event.dart

```
part of 'counter_bloc.dart';

sealed class CounterEvent extends Equatable {
  const CounterEvent();

  @override
  List<Object?> get props => [];
}

final class CounterIncrementPressed extends CounterEvent {}

final class CounterDecrementPressed extends CounterEvent {}

final class CounterResetPressed extends CounterEvent {}
```

Step 2: Define States

lib/bloc/counter_state.dart

```
part of 'counter_bloc.dart';

sealed class CounterState extends Equatable {
  final int count;
  const CounterState({this.count = 0});

  @override
  List<Object?> get props => [count];
}

final class CounterInitial extends CounterState {
  const CounterInitial() : super(count: 0);
}

final class CounterUpdated extends CounterState {
  const CounterUpdated(int count) : super(count: count);
}
```

Step 3: Implement Logic

lib/bloc/counter_bloc.dart

```
import 'package:bloc/bloc.dart';
import 'package:equatable/equatable.dart';

part 'counter_event.dart';
part 'counter_state.dart';

class CounterBloc extends Bloc<CounterEvent, CounterState> {
  CounterBloc() : super(const CounterInitial()) {
    on<CounterIncrementPressed>((event, emit) {
      emit(CounterUpdated(state.count + 1));
    });

    on<CounterDecrementPressed>((event, emit) {
      emit(CounterUpdated(state.count - 1));
    });

    on<CounterResetPressed>((event, emit) {
      emit(const CounterUpdated(0));
    });
  }
}
```

Step 4: Providing and Consuming in UI

`lib/AppMaterial.dart`

```

import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:flutter_application_1/bloc/counter_bloc.dart';

class AppMaterial extends StatelessWidget {
  const AppMaterial({super.key});

  @override
  Widget build(BuildContext context) {
    // 1. Provide the Bloc to the subtree
    return BlocProvider(
      create: (context) => CounterBloc(),
      child: const MaterialApp(
        debugShowCheckedModeBanner: false,
        home: CounterScreen(),
      ),
    );
  }
}

class CounterScreen extends StatelessWidget {
  const CounterScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Colors.amber,
      body: Center(
        child: Row(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            ElevatedButton(
              onPressed: () => context.read<CounterBloc>().add(CounterIncrementPressed()),
              child: const Icon(Icons.add),
            ),
            const SizedBox(width: 16),
            // 2. Rebuild only the Counter Text upon state updates
            BlocBuilder<CounterBloc, CounterState>(
              builder: (context, state) {
                return Text('${state.count}', style: const TextStyle(fontSize: 28));
              },
            ),
            const SizedBox(width: 16),
            ElevatedButton(
              onPressed: () => context.read<CounterBloc>().add(CounterDecrementPressed()),
              child: const Icon(Icons.remove),
            ),
          ],
        ),
      ),
    );
  }
}

```

4. Core Flutter BLoC Widgets

Widget	Description	Primary Use Case
<code>BlocProvider</code>	Provides an instance of a Bloc down the widget tree. Handles lifecycle & auto-close.	Root screens, route definitions, or dependency injection.
<code>BlocBuilder</code>	Rebuilds its child UI in response to state transitions.	Pure UI rendering based on state snapshots.
<code>BlocListener</code>	Invokes a one-time callback whenever state changes (does NOT rebuild UI).	Navigation, SnackBars, Dialogs, Analytics tracking.
<code>BlocConsumer</code>	Combines both <code>BlocBuilder</code> and <code>BlocListener</code> into a single widget.	When a screen needs to rebuild AND execute side effects (e.g. show alert on failure).
<code>BlocSelector</code>	Filters rebuilds based on a specific subsection or property of state.	Optimizing high-frequency re-renders in large forms or lists.

Context Extensions

- `context.read<T>()` : Obtains the Bloc instance without subscribing to changes. Always use inside event handlers and callbacks (e.g., `onPressed`).
- `context.watch<T>()` : Obtains the Bloc and subscribes to rebuild the enclosing widget on every state change.
- `context.select<T, R>()` : Subscribes only to changes in a selected sub-value `R` .

5. BLoC vs Cubit Comparison

Feature	Cubit	BLoC
Trigger Method	Direct functions (<code>cubit.increment()</code>)	Events via stream (<code>bloc.add(Event())</code>)
Boilerplate	Minimal (only State classes)	Moderate (Event classes + State classes)
Event Log / Trace	Tracks state transitions only	Full Event-to-State audit trail
Advanced Async	Manual logic	Transformers (debounce, throttle, switchMap)
Recommended For	Simple UI logic, forms, toggles	Complex domains, event sourcing, real-time queues

6. Golden Rules & Best Practices

Critical Rules to Memorize

- **Never use `context.watch()` inside button callbacks:** Always use `context.read()` . Calling `context.watch()` inside callbacks throws a runtime error.
- **Never mutate states directly:** States must always be immutable. Always emit a new instance (e.g., `emit(CounterUpdated(state.count + 1))`), never mutate fields in-place.
- **Keep builders pure:** Never navigate or show snackbars inside `BlocBuilder.builder` . Route and side-effect logic belongs exclusively in `BlocListener.listener` .
- **Automatic Disposal:** Always instantiate blocs using `BlocProvider(create: (ctx) => MyBloc())` so Flutter automatically disposes them when navigating away.